PROJECT REPORT

LuxuryStay HMS

Hotel Management System

A MERN-stack web application for hotel reservations, front-desk operations, billing, housekeeping, maintenance and management reporting

Submitted by

1	Student1561115	**MUHAMMAD TALHA**
2	Student1561444	**RAJA HARIS**
3	Student1572235	**NIBRAS KHAN**
4	Student1547456	**AYESHA**
5	Student1550000	**MUHAMMAD AHMED**
6	Student1551197	**YASH RAJ**

Academic submission

Institution	_____
Department	_____
Programme / course	_____
Supervisor	_____
Academic session	_____
Submission date	_____

Documentation prepared from the project source code and the running application, 14 October 2026.

Project Team and Declaration

Project team

No.	Student ID	Name	Signature
1	Student1561115	MUHAMMAD TALHA	
2	Student1561444	RAJA HARIS	
3	Student1572235	NIBRAS KHAN	
4	Student1547456	AYESHA	
5	Student1550000	MUHAMMAD AHMED	
6	Student1551197	YASH RAJ	

Declaration

We declare that this project report and the software it describes, **LuxuryStay HMS — Hotel Management System**, are our own work, carried out as part of our academic programme. The work of others, including open-source libraries and their documentation, is acknowledged in the text and the references. This work has not been submitted for any other award.

Date: _____

This declaration is a template; adjust it to the wording your institution requires.

Acknowledgement and Abstract

Acknowledgement

We thank our supervisor, _____, and the faculty of the _____ department of _____ for their guidance, and the maintainers of the open-source tools on which LuxuryStay HMS is built.

Abstract

Hotels must keep reservations, room status, guest identity, payments and housekeeping in step. When these live in registers, spreadsheets and separate tools, rooms are double-booked, balances are missed and managers lack a reliable view of the business.

LuxuryStay HMS is a web-based hotel management system built on the MERN stack: MongoDB with Mongoose, Express 5 on Node.js, and a React 19 single-page application built with Vite. It provides a public website with live room availability, a guest portal for reservation requests with identity-document upload, and a staff workspace for the front desk, housekeeping, management and administration. Five roles are mapped to 29 named permissions enforced on every one of the 59 API endpoints. Double booking is prevented by a unique database index on each room-night, identity documents are encrypted with AES-256-GCM, and a local OCR engine compares a typed CNIC number with the uploaded image as a project-level check only — every online request is confirmed by staff.

The backend has 103 automated tests in 12 suites, run on an isolated database, and the main user journeys were tested end to end in a browser. This report describes the requirements, architecture, data model, roles, workflows and security of the system, how to install and run it, and the verified test results.

Keywords: hotel management system, MERN stack, role-based access control, reservations, room-night ledger

Table of Contents

1 Introduction and Objectives 1
1.1 Problem 1
1.2 Aim and objectives 1
2 Scope and Requirements 2
2.1 Scope 2
2.2 Functional requirements 2
2.3 Non-functional requirements 2
3 Technology Stack 3
4 System Architecture 4
5 User Roles and Permissions 6
6 Database Design 9
6.1 Integrity mechanisms 9
7 System Modules 11
8 Authentication, Authorization, Validation and Security 12
8.1 Authentication and authorization 12
8.2 Validation rules 12
8.3 Request, data and database protection 12
9 Core Hotel Workflows 14
10 API Structure 16
11 Testing and Results 17
11.1 Strategy 17
11.2 Results (15 October 2026) 17
12 Application Screenshots 19
13 Installation and Configuration 21
14 Known Limitations and Future Improvements 22
15 Conclusion and References 23
References 23
Appendix A API Endpoints 24

1 Introduction and Objectives

LuxuryStay HMS is a full-stack web application that supports the daily work of a single hotel, from a visitor's first look at the website to the guest's departure and review. It brings reservations, room status, guest records, identity documents, billing, housekeeping, maintenance and reporting into one system used through a browser by guests and by every member of staff.

The application has three parts: a **public website** that presents the room types with live prices and availability; a **guest portal** where a registered guest requests and manages stays; and a **staff workspace** whose screens and actions depend on the person's role.

1.1 Problem

- Rooms can be promised twice when availability is checked by reading a register or spreadsheet.
- Identity documents are checked inconsistently and copies are stored without protection.
- Folios — room charges, extras, discounts, tax and part-payments — are calculated by hand.
- Reception does not know when housekeeping has finished a room, and maintenance issues are forgotten.
- Managers have no live view of occupancy, revenue or guest satisfaction, and access is not controlled per person.

1.2 Aim and objectives

Aim: to design and implement a secure, role-based web application that manages a hotel's reservations, stays, billing, housekeeping, maintenance and reporting, with a public website and a guest portal.

1. Present room types with real prices and availability for chosen dates.
2. Let visitors register as guests and request reservations online with an identity document.
3. Prevent double booking under concurrent requests by enforcing availability in the database.
4. Support the front-desk cycle: desk reservations, review of requests, check-in, payments, charges, discounts, invoices and check-out.
5. Connect check-out to housekeeping and track maintenance that takes rooms out of service.
6. Give managers live dashboards and period reports.
7. Restrict every action to the roles allowed to perform it, on the server.
8. Protect accounts and personal data, and keep production, development and test data apart.

This report describes the system exactly as implemented; every technical statement was checked against the source code, and features that do not exist are listed as limitations.

2 Scope and Requirements

2.1 Scope

In scope: one hotel with three room categories (Serene Room, Signature Suite, Royal Residence); public website, guest portal and staff workspace; online and desk reservations with CNIC or passport; check-in and check-out; folios, payments and numbered invoices; housekeeping, maintenance, reviews, enquiries and in-app notifications; staff accounts, settings, dashboards and reports; password reset by email.

Out of scope: online card payments, official identity verification (NADRA or any government service), multiple properties or channel managers, restaurant and spa point-of-sale, and SMS or email notifications other than password reset.

2.2 Functional requirements

Table 2.1: Functional requirements (all implemented)

ID	Requirement
FR-1	Show room types with prices, amenities and capacity from the database; check availability for dates and guests.
FR-2	Register guest accounts; sign in and out; update profile and password; reset a forgotten password by email.
FR-3	Lock an account for 15 minutes after 8 failed sign-ins; refuse deactivated accounts.
FR-4	Guests request stays with an identity document (CNIC front and back, or passport); requests stay pending until staff decide.
FR-5	Staff create confirmed reservations at the desk; confirm or reject requests (rejection needs a reason).
FR-6	Change dates or room; guests may change only pending requests; cancel pending or confirmed reservations.
FR-7	Never let two active reservations hold the same room on the same night.
FR-8	Check in confirmed, verified reservations from the arrival day; check out when the folio is settled.
FR-9	Maintain folios: room charges, extras, discount, tax, payments (cash, card, bank transfer); issue numbered invoices.
FR-10	Manage rooms and room status; run the cleaning queue; report, start and complete maintenance.
FR-11	Guests review completed stays; managers moderate reviews; staff handle website enquiries.
FR-12	Dashboards and reports for 7, 30, 90 days or 12 months.
FR-13	Administrators create, edit, activate and deactivate staff accounts and change hotel settings.

2.3 Non-functional requirements

Table 2.2: Non-functional requirements

Area	How it is met
Security	bcrypt password hashes; revocable JWT sessions; permission check on every protected endpoint; rate limits; strict input schemas; security headers; encrypted identity documents.
Integrity	Unique (room, night) index; optimistic concurrency on bookings; atomic invoice numbers; start-up checks of database name, indexes and ledger.

Area	How it is met
Usability and accessibility	Role-specific navigation; field-level validation messages; light and dark themes; reduced-motion support; layouts from 390 px phones to desktops.
Maintainability	Routes, middleware, services, models and utilities separated; one permission matrix; 12 automated test suites.

3 Technology Stack

Table 3.1: Technology stack (versions from package.json; runtime versions as tested)

Layer	Technologies
Runtime and database	Node.js 24.15.0; MongoDB 8.2.12 (standalone)
Backend	bcryptjs ^3.0.3, cors ^2.8.6, dotenv ^18.0.0, express ^5.2.1, express-rate-limit ^8.7.1, helmet ^8.3.0, jsonwebtoken ^9.0.3, mongoose ^9.10.1, multer ^2.4.0, nodemailer ^10.1.0, tesseract.js ^5.1.1, zod ^4.6.7
Frontend	axios ^1.20.0, framer-motion ^13.4.0, lucide-react ^1.47.0, react ^19.2.8, react-dom ^19.2.8, react-icons ^5.7.0, react-router-dom ^7.18.4
Tooling	@types/react ^19.2.18, @types/react-dom ^19.2.7, @vitejs/plugin-react ^6.1.1, oxlint ^1.81.0, vite ^8.3.0; nodemon ^3.1.14; Node test runner

`axios` and `react-icons` are declared in the client but not imported anywhere.

4 System Architecture

LuxuryStay HMS has three tiers. The React application runs in the browser and calls the Express API over HTTP with JSON; the API is the only component that reads or writes MongoDB, so every rule — who may act, which nights are free, how a folio is totalled — is enforced in one place (Figure 4.1).

- **Client:** `App.jsx` defines the public website (`/`, `/rooms/:slug`), authentication pages, the guest portal (`/guest`) and the staff workspace (`/admin` for administrators and managers, `/staff` for receptionists and housekeeping). A signed-in person opening another role's area is redirected to the same page in their own area. All requests go through `lib/api.js`.
- **API:** a fixed middleware pipeline — security headers, CORS, body limit, unsafe-key rejection, rate limits, database-ready guard — then authentication, permission check, schema validation and the handler. Shared business rules live in `services/` (availability, identity ledger, OCR, invoices, notifications, settings).
- **Data:** MongoDB with 12 collections; identity-document images are stored encrypted in the upload folder. Separate databases are used for production (`luxurystay`), development (`luxurystay_dev`) and tests (`<name>_test`).

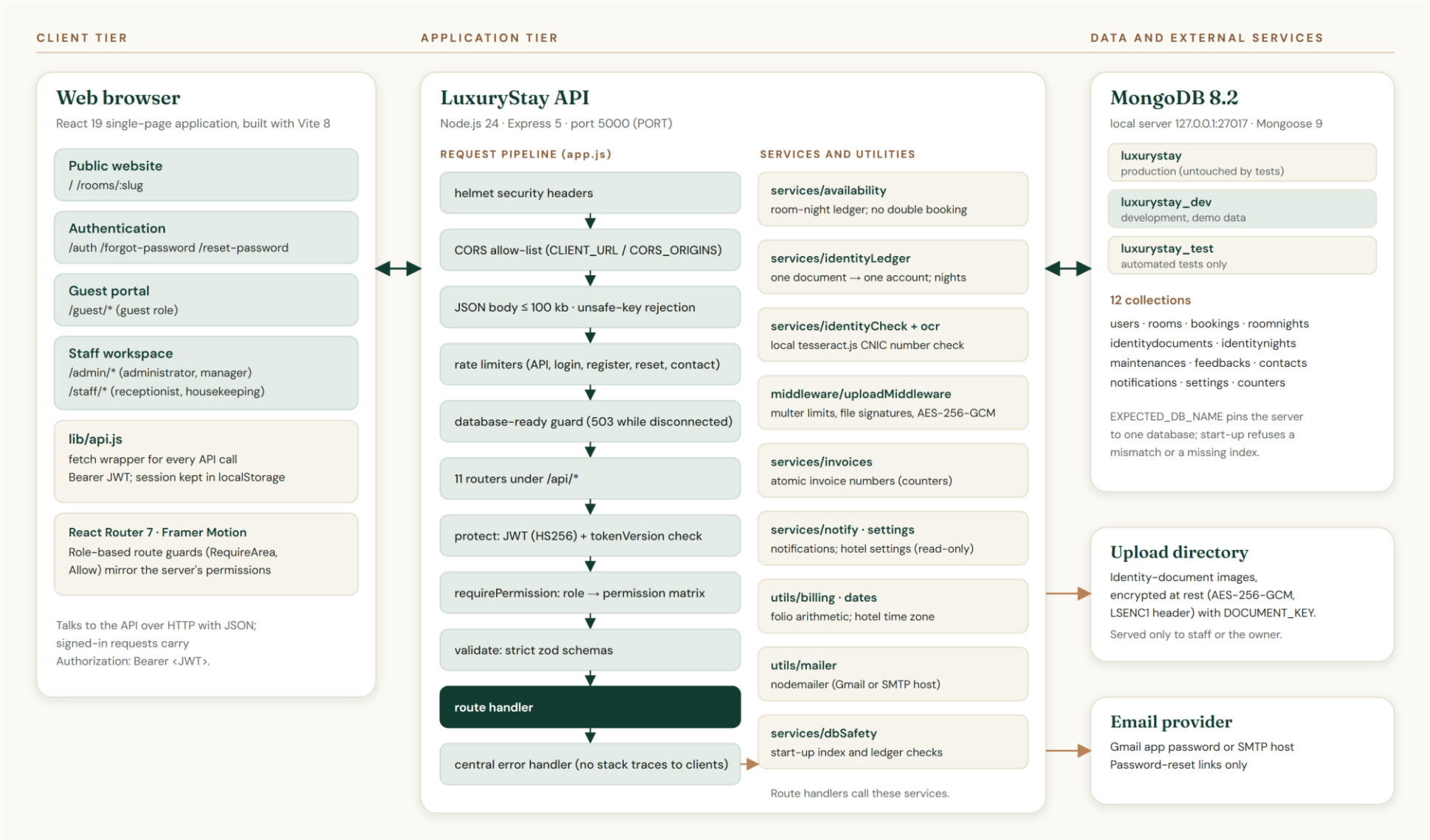


Figure 4.1: System architecture

5 User Roles and Permissions

Table 5.1: The five roles

Role	Workspace	Responsibilities
Administrator (admin)	/admin	Everything a manager can do, plus staff accounts, guest-account activation and hotel settings
Manager	/admin	Front desk and housekeeping operations, rooms, discounts, reports, reviews; can view the staff list
Receptionist	/staff	Reservations, identity review, check-in and check-out, payments and invoices, enquiries, guest directory
Housekeeping	/staff	Cleaning queue (rooms they start are marked as theirs), maintenance tasks; no guest, booking or billing data
Guest	/guest	Own reservations, invoices, reviews and profile

Registration always creates a guest. Staff accounts are created only by an administrator; nobody can change their own role or status, and the last active administrator cannot be removed. A legacy `staff` role remains for older accounts but cannot be assigned.

Table 5.2: Permission matrix (generated from server/config/permissions.js)

Permission	Meaning	admin	manager	receptionist	housekeeping	guest
`staff:view`	View staff accounts	•	•			
`staff:manage`	Create and edit staff accounts	•				
`guests:view`	View the guest directory	•	•	•		
`guests:manage`	Activate or deactivate guest accounts	•				
`rooms:view`	View all rooms and their status	•	•	•	•	
`rooms:manage`	Add, edit and retire rooms	•	•			
`rooms:status`	Change a room's status	•	•	•		
`housekeeping:manage`	Work the cleaning queue	•	•		•	
`maintenance:view`	View maintenance tasks	•	•	•	•	
`maintenance:report`	Report a maintenance issue	•	•	•	•	
`maintenance:update`	Start and complete maintenance	•	•		•	
`bookings:own`	Make and manage own reservations					•
`bookings:view`	View all reservations	•	•	•		
`bookings:create`	Create reservations at the desk	•	•	•		
`bookings:review`	Confirm or reject requests	•	•	•		
`bookings:edit`	Change reservation dates or room	•	•	•		

Permission	Meaning	admin	manager	receptionist	housekeeping	guest
`bookings:cancel`	Cancel any reservation	•	•	•		
`bookings:checkin`	Check guests in	•	•	•		
`bookings:checkout`	Check guests out	•	•	•		
`billing:view`	View folios and invoices	•	•	•		
`billing:payment`	Record payments	•	•	•		
`billing:charges`	Add or remove extra charges	•	•	•		
`billing:discount`	Apply discounts	•	•			
`reports:view`	View dashboard reports	•	•			
`frontdesk:view`	View the front-desk summary	•	•	•		
`feedback:own`	Review own completed stays					•
`feedback:manage`	Moderate guest reviews	•	•			
`contact:manage`	Handle website enquiries	•	•	•		
`settings:manage`	Change hotel settings	•				

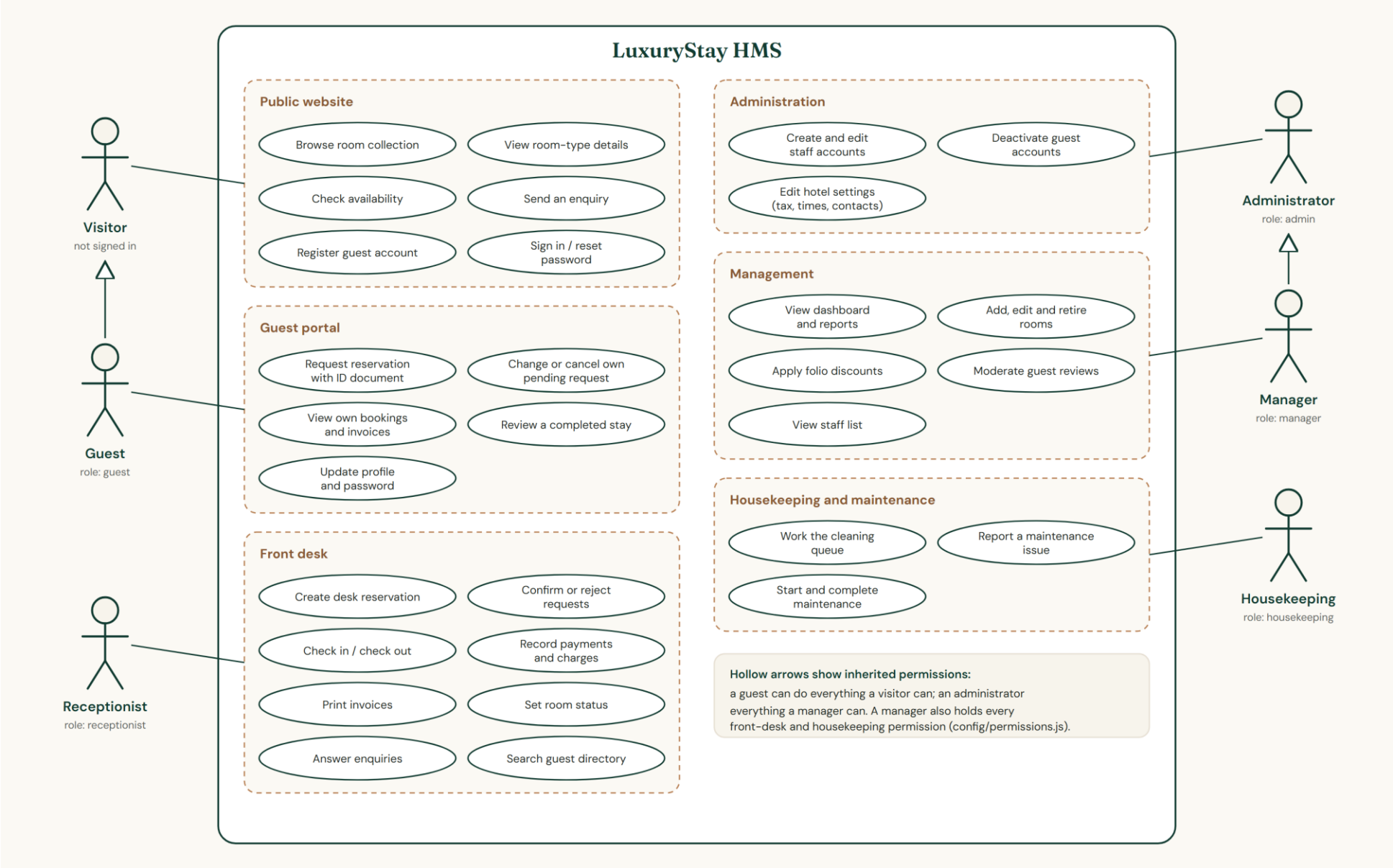


Figure 5.1: Use case diagram

6 Database Design

Each Mongoose model in `server/models` maps to one MongoDB collection; references are stored as ObjectIds (Figure 6.1).

Table 6.1: Collections

Collection	Purpose	Key constraints
`users`	Accounts, role, sign-in security fields	email unique
`rooms`	Room number, type, rate, capacity, status, housekeeping fields	roomNumber unique
`bookings`	Stay, guest details, identity check, folio (charges, payments), audit fields	invoiceNumber unique when set; version-checked saves
`roomnights`	One document per room per night held by an active booking	(room, night) unique
`identitydocuments`	Binds a normalised document number to one account	_id is the document key
`identitynights`	Nights held by a document	(key, night) unique
`maintenances`	Issues, priority, progress and resolution	one open task per room (checked in handler)
`feedbacks`	Reviews of completed stays	booking unique
`contacts`	Website enquiries	—
`notifications`	In-app notifications for a user or roles	expire after 60 days
`settings`	Hotel-wide settings	key unique
`counters`	Atomic invoice sequences	—

6.1 Integrity mechanisms

- **Room-night ledger:** an active reservation inserts one `roomnights` document per night; the unique index makes MongoDB refuse a second claim, so double booking cannot happen even under simultaneous requests and without transactions.
- **Identity ledger:** a document number belongs to one account and holds each night once, unless the front desk approves a second room for the same guest.
- **Optimistic concurrency** on bookings and **atomic invoice numbers** from `counters`.
- **Start-up checks:** the server verifies the database name (`EXPECTED_DB_NAME`), required indexes and ledger consistency before serving requests.

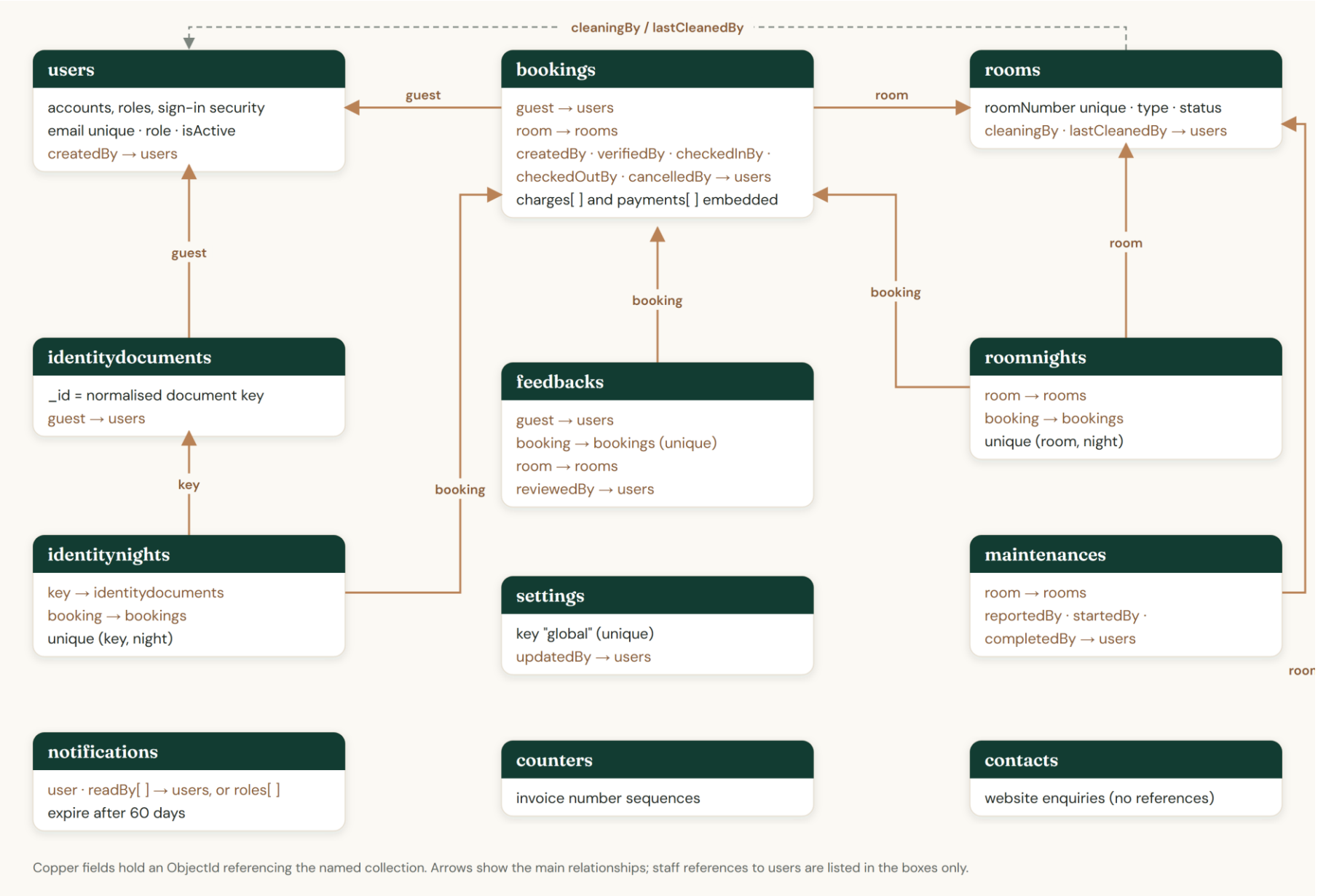


Figure 6.1: Collections and their references

7 System Modules

Table 7.1: Implemented modules

Module	What it does
Public website	Room types from GET /api/public/rooms (no room numbers or status exposed), availability search, room pages, enquiry form.
Accounts	Registration (guests only), sign-in, sign-out (token revoked), profile, password change, password reset by email.
Guest portal	Overview of the next stay; five-step booking (dates, room, details, identity, review); My bookings with details, invoices, changes or cancellation of pending requests, reviews.
Reservations	List with filters and search; detail drawer with the actions the role and status allow; desk reservations confirmed immediately.
Front desk	Arrivals and departures; check-in requires a verified booking, the arrival day and an available room; check-out requires a settled folio unless settings allow a balance; the room then goes to cleaning.
Billing	Folios with room charges, extras, manager discounts and tax; payments with method and reference (duplicate submissions applied once); printable invoices.
Rooms and housekeeping	Room inventory and status; cleaning queue where a housekeeper starts a room and marks it clean.
Maintenance	Issues with priority; reporting takes a room out of service (not while occupied); completion with a note returns it.
Reports	Dashboard and period reports: revenue, occupancy, reservations, average nightly rate, rating; printable.
Reviews and enquiries	One review per completed stay; moderation; enquiries marked read or resolved.
Administration	Staff accounts with search and role/status filters; guest-account activation; hotel settings.
Identity documents	CNIC or passport upload (JPG, PNG, WebP or PDF, 5 MB, 2 files), file-signature check, local OCR comparison for CNIC, AES-256-GCM encryption.

8 Authentication, Authorization, Validation and Security

8.1 Authentication and authorization

- Passwords hashed with bcrypt (12 rounds); never returned by the API.
- JWT (HS256, 12 h by default) carrying a token version: signing out, changing or resetting a password, a role change or deactivation ends every session; each request reloads the account.
- 8 failed sign-ins lock the account for 15 minutes; reset links are single-use, stored only as a hash and expire after 30 minutes; the reset response does not reveal whether an email exists.
- Every protected route names a permission; handlers also check ownership so a guest reaches only their own bookings, invoices and documents. Unauthorized requests receive 401 or 403 — hiding a button is never the only protection.

8.2 Validation rules

The client and the server apply the same rules; the server always validates again with strict zod schemas that reject unknown fields.

Table 8.1: Validation rules

Field	Rule
Account name (registration, profile, staff, create-admin)	3–80 characters after trimming, starting with a letter; letters, numbers, spaces and . ' ’ - only. 1 or 2 characters, or only spaces, give “Name must be at least 3 characters.”
Guest name on a reservation, enquiry name	2–80 characters, same character rule (real names can have two letters)
Email	Valid address, stored in lower case, unique per account (duplicate → 409)
Password	8–72 characters with a letter and a number; confirmation must match (client)
Phone / CNIC / passport	7–15 digits / 13 digits with or without dashes / 6–12 letters and digits
Stay	Check-out after check-in, not in the past, within the maximum stay and advance period and the room's capacity

There is no separate username: people sign in with their email address, and the Full name field is the account name. Before this release the account name accepted two characters; the three-character minimum was added on both the server (`accountName` schema) and the client (`rules.accountName`) with regression tests (Chapter 11).

8.3 Request, data and database protection

- Rate limits: 1,500 API requests, 10 failed sign-ins, 5 reset requests and 15 reset-token checks per 15 minutes; 10 registrations and 8 enquiries per hour.
- Security headers (helmet), CORS allow-list, 100 kb JSON limit, rejection of operator-injection keys, no stack traces in responses.

- Identity documents are encrypted with AES-256-GCM using `DOCUMENT_KEY`; without the key, uploads are refused (503) — nothing is stored unencrypted. Documents are served only to front-desk staff or the owner, and each viewing is logged without content.
- Secrets live only in `server/.env`, which is excluded from Git along with uploads and database exports.
- Maintenance scripts print the connected database and refuse the production and test databases; tests refuse any database whose name does not end in `_test`.

> **Identity checking**
>
> The CNIC check compares the typed number with the number read from the image by a local OCR engine. It is a project-level consistency check, is not connected to NADRA, never confirms a booking by itself, and approving a mismatch requires a staff note.

9 Core Hotel Workflows

- **Online request:** a guest requests a stay; the nights are held at once (pending); staff confirm after reviewing the document or reject with a reason.
- **Desk reservation:** staff book a walk-in or telephone guest; the document is inspected in person, so the reservation is confirmed and receives an invoice number.
- **Stay:** check-in from the arrival day marks the room occupied; extras, discounts and payments update the folio; check-out (balance settled) sends the room to cleaning.
- **Housekeeping:** the room is started (marked as that housekeeper's) and marked clean, which makes it available again.
- **Cancellation:** pending or confirmed reservations can be cancelled, releasing the nights; a guest cannot cancel once a payment is recorded.

Table 9.1: Room status changes

From	Event	To
available	Guest checks in	occupied
occupied	Guest checks out	cleaning
cleaning	Housekeeping marks the room clean	available
any except occupied	Maintenance issue reported	maintenance
maintenance	Maintenance completed	available

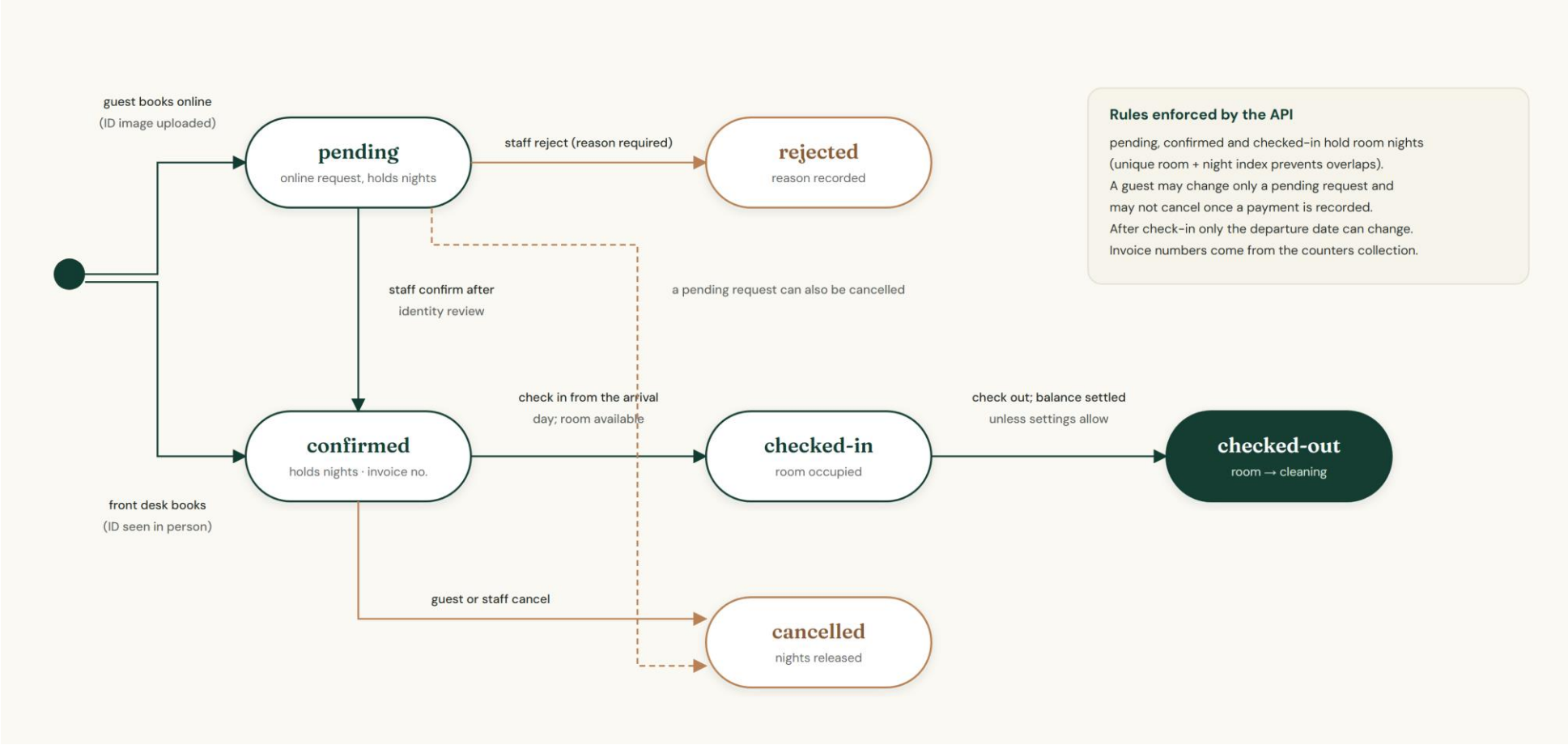


Figure 9.1: Reservation states and transitions

10 API Structure

The API exposes 59 endpoints under `/api` (plus `GET /api/health`) in 11 routers. Requests and responses are JSON, except reservation requests with documents (multipart). Signed-in requests send `Authorization: Bearer <token>`; errors return `success: false` with a message, and validation errors list the fields concerned. Appendix A lists every endpoint with its access rule.

Table 10.1: API routers

Base path	Endpoints	Responsibility
`/api/auth`	9	Registration, sign-in, profile, password change and reset
`/api/bookings`	17	Reservations, availability, check-in/out, payments, charges, discounts, invoices, documents
`/api/contact`	3	Website enquiries
`/api/feedback`	4	Guest reviews
`/api/maintenance`	4	Maintenance tasks
`/api/notifications`	3	In-app notifications
`/api/public`	2	Hotel details and room types for the website
`/api/reports`	3	Dashboard, front-desk and finance figures
`/api/rooms`	8	Rooms and housekeeping
`/api/settings`	2	Hotel settings
`/api/users`	4	Staff accounts and guest directory

11 Testing and Results

11.1 Strategy

- **Automated backend tests** (Node test runner): each suite starts the real Express application against a disposable database, creates its own data and checks responses and stored documents; OCR and email are replaced by test doubles. For this report the suite ran on `lsverify_test`, so neither `luxurystay_test` nor the development data was touched.
- **Browser end-to-end tests** (headless Microsoft Edge) on an isolated stack — frontend on port 5199, test API on port 5055, database `lsverify_test` seeded with fictional data — covering every role's main journeys and API refusals.
- **Development-stack checks** on `luxurystay_dev`: each role's demo account signed in; nothing else was written.
- **Static checks:** client lint (oxlint) and production build (Vite).

Table 11.1: Backend test suites

Suite	Tests	Covers
`auth.test.js`	16	Registration, sign-in, lockout, tokens, profile, security headers
`bookings.test.js`	13	Availability, overlap prevention, concurrency, folios
`create-admin.test.js`	5	Database guards of create-admin and the seed scripts
`db-safety.test.js`	5	Test-database naming, index and ledger checks, start-up refusal
`identity.test.js`	15	Uploads, OCR outcomes, encryption, staff review rules
`ocr-engine.test.js`	1	Real OCR on a fictional specimen card
`operations.test.js`	14	Check-in/out, housekeeping, maintenance, payments, invoices, reports
`password-reset.test.js`	10	Reset tokens, single use, rate limits, sign-out everywhere
`public.test.js`	3	Public hotel and room endpoints
`rbac.test.js`	12	Role permissions, IDOR, staff guard rails
`registration.test.js`	5	Account-name minimum of 3 characters, invalid names, duplicates
`roles.test.js`	4	Five roles, staff creation, permissions of new staff, filters

11.2 Results (15 October 2026)

Table 11.2: Verified results

Check	Result
Backend suite (`npm test`) on lsverify_test	103 tests: 103 passed, 0 failed, 0 skipped. Document counts of luxurystay_dev and luxurystay_test identical before and after.
Registration regression tests	5 of 5 passed: names of 1 and 2 characters and blank names rejected with the 3-character message; 3+ characters accepted and trimmed; invalid characters rejected; duplicate email → 409; profile and staff names use the same rule.

Check	Result
Browser: public site and accounts	Room pages and images load (no broken images); registration validation for 1, 2 and 3-character names, invalid characters, password mismatch and invalid email; registration, duplicate refusal, sign-in, wrong password, sign-out; password recovery refused safely with email switched off.
Browser: workflows	Guest booking with a passport image (stored encrypted, LSENC1), staff confirmation, desk reservation, check-in, payment, check-out, cleaning, maintenance completion, room creation, review moderation, enquiry handling, staff creation, edit and deactivation (sign-in then refused), settings change, guest review and cancellation — all completed with the expected messages. Three steps failed first because of wrong labels in the test script; they were corrected and passed.
Browser: authorization	Receptionist, housekeeping, manager and guest received 403 for every endpoint outside their permissions that was tried (staff, settings, reports, bookings, guests, housekeeping).
Responsive layout	13 pages across four roles at 390 px and 820 px: no horizontal overflow, no console errors.
Data consistency after the tests	No missing indexes; room-night ledger complete; occupied rooms match in-house guests; payment totals consistent.
Client lint and build	0 errors, 31 warnings (react set-state-in-effect); production build succeeded.

Not covered: Firefox, Safari and real phones; load testing; automated frontend unit tests.

12 Application Screenshots

Screenshots were captured from the running application with fictional demonstration data: development database `luxurystay_dev` (Figures 12.1–12.16) and the isolated test environment `lsverify_test` (Figures 12.17–12.20), at 1440 × 900 unless noted. Account lists are filtered to demonstration accounts; no passwords, tokens or real documents are shown.

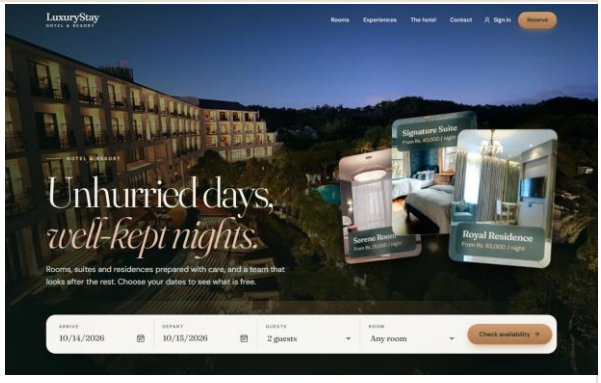


Figure 12.1: Public home page

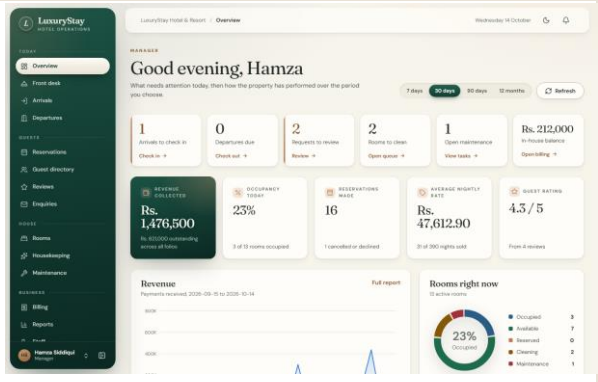


Figure 12.2: Management dashboard

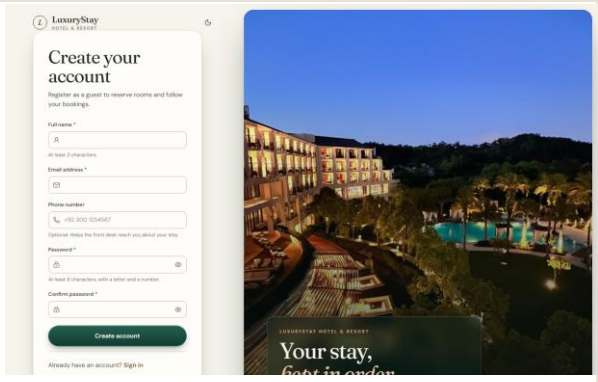


Figure 12.3: Guest registration

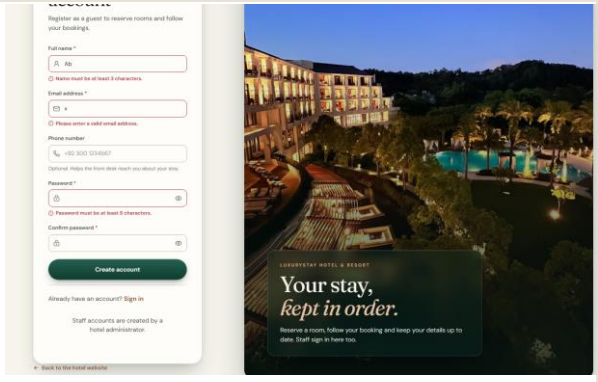


Figure 12.4: Registration validation: 3-character minimum

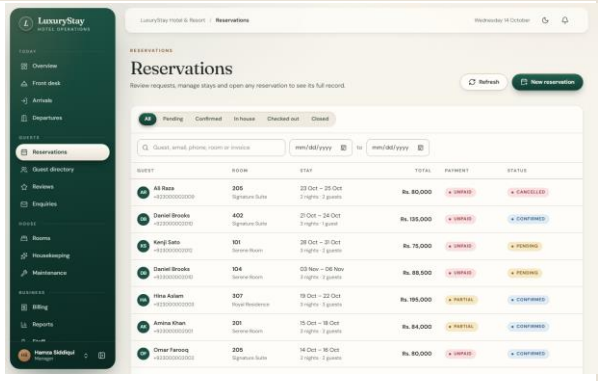


Figure 12.5: Reservations

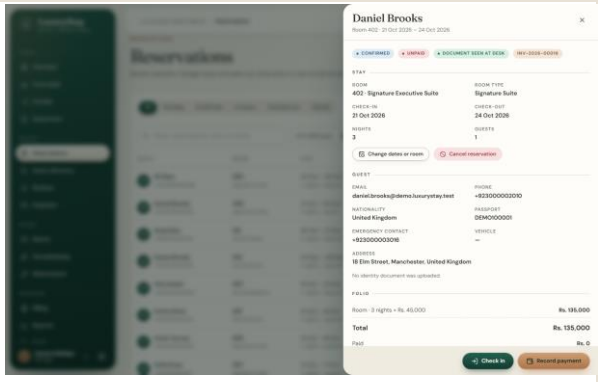


Figure 12.6: Reservation details and folio

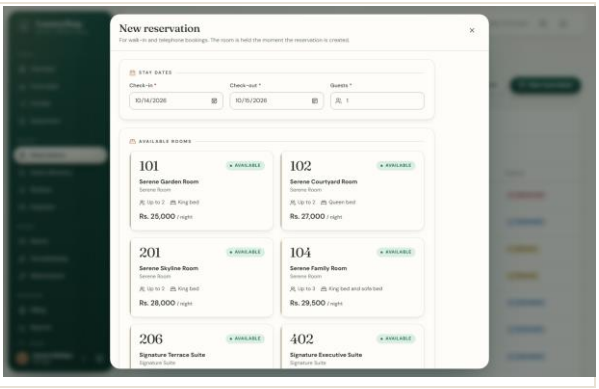


Figure 12.7: Desk reservation form

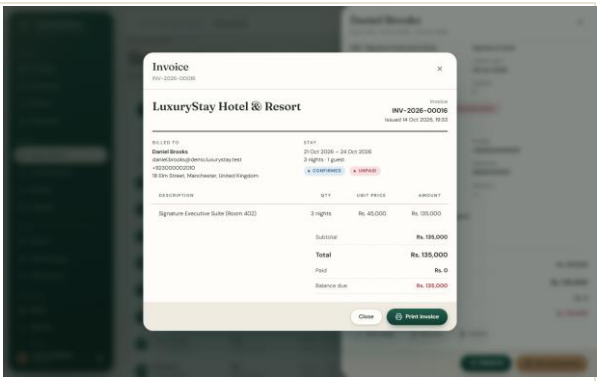


Figure 12.8: Invoice

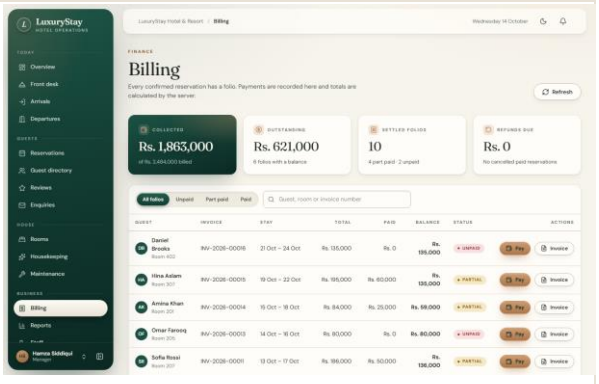


Figure 12.9: Billing

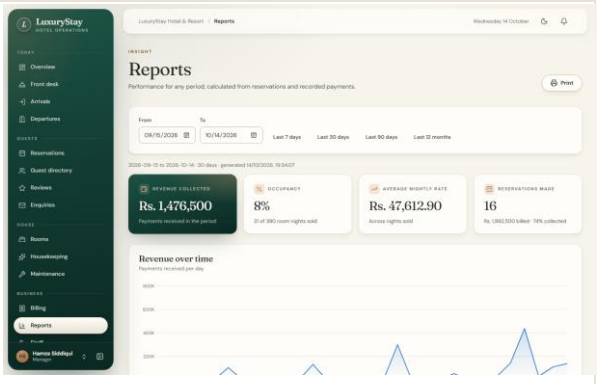


Figure 12.10: Reports

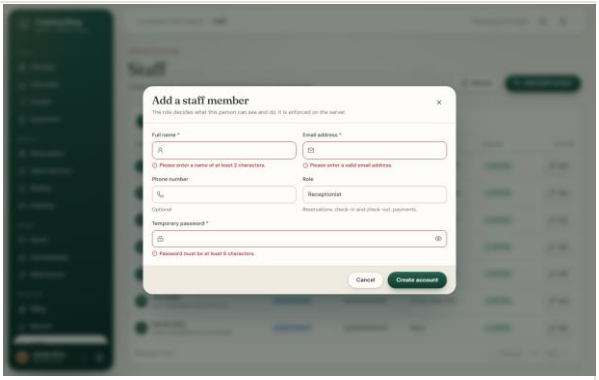


Figure 12.11: Add staff member (administrator)

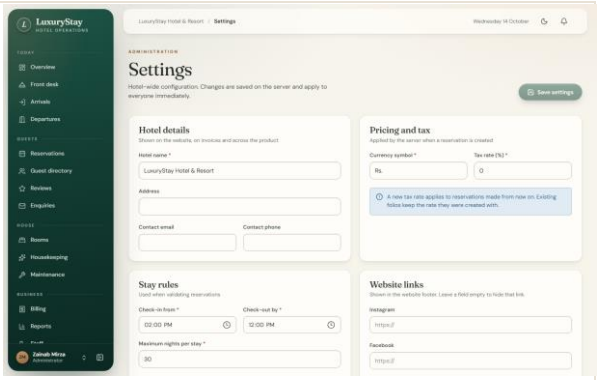


Figure 12.12: Hotel settings (administrator)

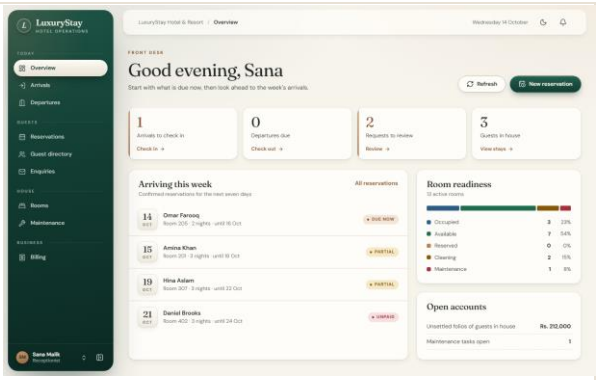


Figure 12.13: Receptionist overview

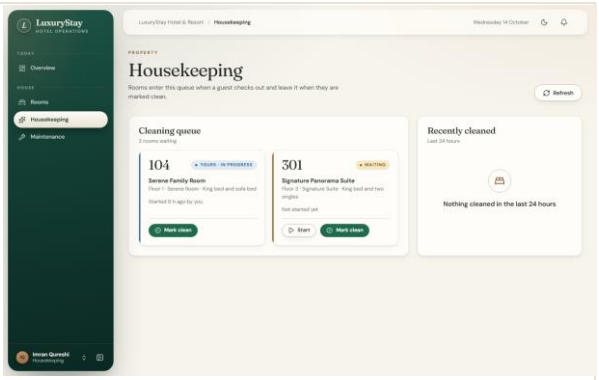


Figure 12.14: Housekeeping queue: "Yours"

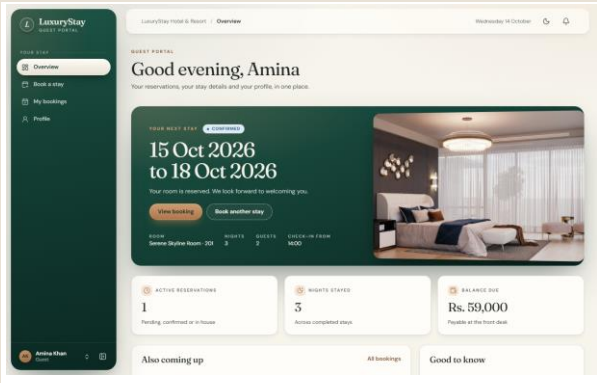


Figure 12.15: Guest overview

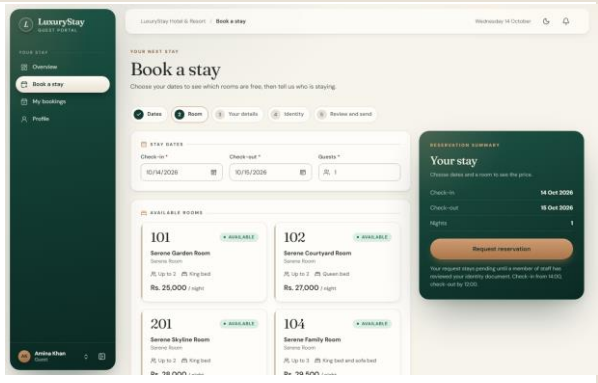


Figure 12.16: Guest booking flow

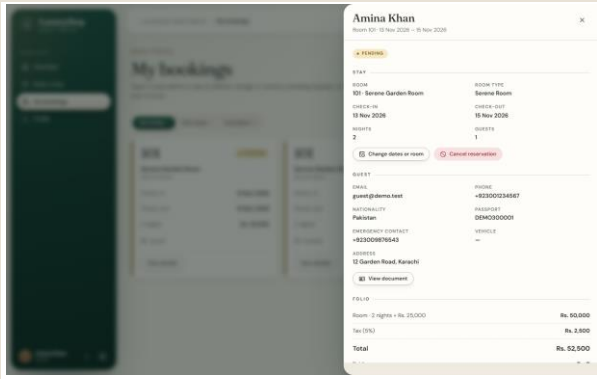


Figure 12.17: Guest request with uploaded document (test environment)

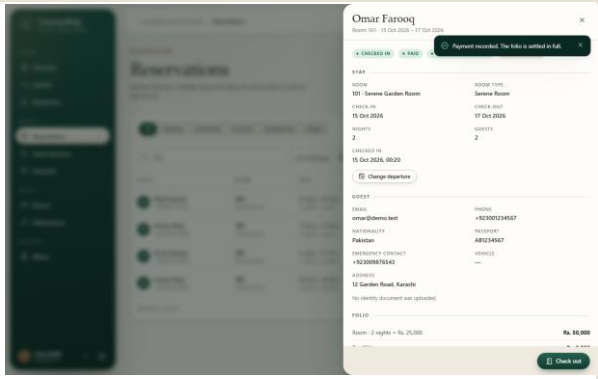


Figure 12.18: Payment recorded, folio settled (test environment)

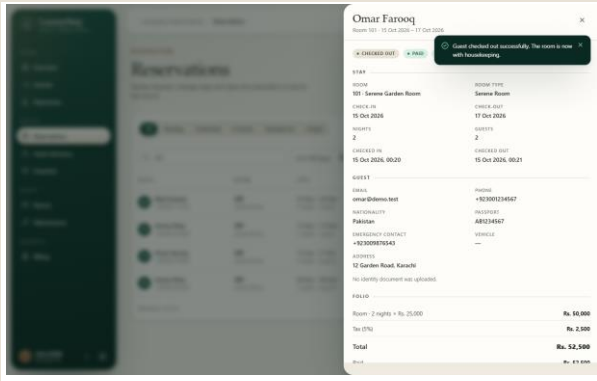


Figure 12.19: Guest checked out; room to housekeeping (test environment)

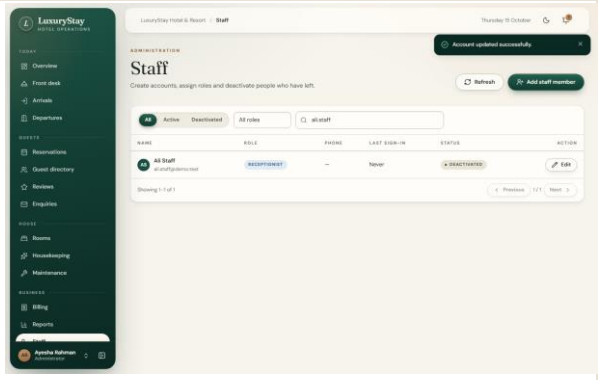


Figure 12.20: Staff account edited and deactivated (test environment)

13 Installation and Configuration

1. Install Node.js 24 and MongoDB 8; run `npm install` in `server` and in `client`.
2. Copy `server/.env.example` to `server/.env` and set `MONGO_URI` (with the database name, e.g. `luxurystay_dev`), `EXPECTED_DB_NAME`, a 32+ character `JWT_SECRET`, and optionally `EMAIL_USER`/`EMAIL_APP_PASSWORD` (password reset) and `DOCUMENT_KEY` (document uploads).
3. Create the required indexes: `npm run db:indexes -- --confirm=luxurystay_dev`.

4. Create the first administrator: set `ADMIN_EMAIL`, `ADMIN_NAME` and `ADMIN_PASSWORD`, then `npm run create-admin` (refuses the production and test databases).
5. Optional demo data: set `SEED_DEMO_PASSWORD` and run `npm run seed:dev-demo` (development database only; adds missing records only).
6. Start the API with `npm run dev` in `server` (http://localhost:5000) and the website with `npm run dev` in `client` (http://localhost:5173); `GET /api/health` reports the connected database.
7. Run the tests on a throwaway database: `$env:MONGO_URI = "mongodb://127.0.0.1:27017/lsverify"; npm test`.

Secrets are generated with `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"` and kept only in `server/.env`. The root README.md gives the full variable list, the demo accounts and troubleshooting.

14 Known Limitations and Future Improvements

Table 14.1: Known limitations

Limitation	Effect or note
No payment gateway	Payments are recorded by staff.
Document uploads need DOCUMENT_KEY	Without it, online requests with documents are refused (503).
CNIC check compares numbers only	Cannot detect forged cards; PDFs are not read by OCR; first use downloads OCR data.
Email for password reset only	No booking or invoice emails, no SMS.
Pending requests do not expire	They hold their nights until staff decide; no no-show handling.
Housekeeping has no inspection step or manager assignment	Rooms are claimed by whoever starts them.
No walk-in enquiry form, no full audit log, no report export	Enquiries come from the website; the staff list shows last sign-in only; reports can be printed.
Single hotel and currency; shared photos per room type	Not suited to chains as is.
31 lint warnings; no frontend unit tests	Behaviour verified by browser tests.

Future improvements: payment gateway with refunds; confirmation emails and SMS; expiry of unanswered requests; housekeeping assignment and inspection; per-room photos and seasonal rates; report export; two-factor authentication for staff; automated frontend tests; deployment scripts and backups.

15 Conclusion and References

LuxuryStay HMS meets its aim of a secure, role-based web application for running a hotel. Visitors explore rooms and availability, guests request and manage stays, staff handle the full cycle from reservation to check-out, housekeeping and maintenance, and managers follow performance on live dashboards. Its key design decisions protect correctness and data: availability is enforced by a unique database index, sessions are revocable, every endpoint checks a named permission, identity documents are encrypted and never treated as officially verified, and production, development and test data are kept apart by enforced safeguards. The automated and browser tests reported in Chapter 11 support these claims, and the limitations in Chapter 14 point to clear next steps.

References

1. Node.js documentation and test runner. https://nodejs.org/api/
2. Express 5. https://expressjs.com/ — MongoDB manual. https://www.mongodb.com/docs/manual/ — Mongoose. https://mongoosejs.com/docs/
3. React. https://react.dev/ — Vite. https://vite.dev/ — React Router. https://reactrouter.com/ — Motion. https://motion.dev/
4. Zod. https://zod.dev/ — Helmet. https://helmetjs.github.io/ — express-rate-limit. https://express-rate-limit.mintlify.app/ — Multer. https://github.com/expressjs/multer
5. Jones, M., Bradley, J. and Sakimura, N. (2015) JSON Web Token (JWT), RFC 7519. https://www.rfc-editor.org/rfc/rfc7519
6. Tesseract.js. https://github.com/naptha/tesseract.js — Nodemailer. https://nodemailer.com/
7. OWASP Cheat Sheet Series (Authentication, Password Storage, Forgot Password). https://cheatsheetseries.owasp.org/
8. NIST FIPS 197 (AES) and SP 800-38D (GCM). https://csrc.nist.gov/

Appendix A API Endpoints

All 59 endpoints, extracted from the route files; `GET /api/health` is defined in `app.js` and is public. "Signed in" means the handler checks ownership or role itself.

Table A.1: API endpoints

Method	Path	Access
POST	/api/auth/register	public (rate limit: register)
POST	/api/auth/login	public (rate limit: login)
POST	/api/auth/logout	signed in
GET	/api/auth/me	signed in
PUT	/api/auth/me	signed in
PUT	/api/auth/change-password	signed in
POST	/api/auth/forgot-password	public (rate limit: passwordReset)
POST	/api/auth/reset-password/validate	public (rate limit: resetToken)
POST	/api/auth/reset-password	public (rate limit: resetToken)
GET	/api/bookings/availability	bookings:own or bookings:create or bookings:edit
POST	/api/bookings	bookings:own or bookings:create
GET	/api/bookings/my-bookings	bookings:own
GET	/api/bookings	bookings:view
GET	/api/bookings/:id	signed in
PATCH	/api/bookings/:id	bookings:own or bookings:edit
PATCH	/api/bookings/:id/confirm	bookings:review
PATCH	/api/bookings/:id/reject	bookings:review
PATCH	/api/bookings/:id/cancel	bookings:own or bookings:cancel
PATCH	/api/bookings/:id/check-in	bookings:checkin
PATCH	/api/bookings/:id/check-out	bookings:checkout
POST	/api/bookings/:id/payments	billing:payment
POST	/api/bookings/:id/charges	billing:charges
DELETE	/api/bookings/:id/charges/:chargeId	billing:charges
PATCH	/api/bookings/:id/discount	billing:discount
GET	/api/bookings/:id/invoice	signed in
GET	/api/bookings/:id/documents/:side	signed in
POST	/api/contact	public (rate limit: contact)
GET	/api/contact	contact:manage
PATCH	/api/contact/:id/status	contact:manage
POST	/api/feedback	feedback:own
GET	/api/feedback/my	feedback:own
GET	/api/feedback	feedback:manage
PATCH	/api/feedback/:id/review	feedback:manage
GET	/api/maintenance	maintenance:view
POST	/api/maintenance	maintenance:report

Method	Path	Access
PATCH	/api/maintenance/:id/start	maintenance:update
PATCH	/api/maintenance/:id/complete	maintenance:update
GET	/api/notifications	signed in
PATCH	/api/notifications/read-all	signed in
PATCH	/api/notifications/:id/read	signed in
GET	/api/public/hotel	public
GET	/api/public/rooms	public
GET	/api/reports/overview	reports:view
GET	/api/reports/front-desk	frontdesk:view
GET	/api/reports/finance	billing:view
GET	/api/rooms	signed in
GET	/api/rooms/housekeeping	housekeeping:manage
PATCH	/api/rooms/housekeeping/:id/start	housekeeping:manage
PATCH	/api/rooms/housekeeping/:id/complete	housekeeping:manage
POST	/api/rooms	rooms:manage
PUT	/api/rooms/:id	rooms:manage
PATCH	/api/rooms/:id/status	rooms:status
DELETE	/api/rooms/:id	rooms:manage
GET	/api/settings	signed in
PUT	/api/settings	settings:manage
GET	/api/users/staff	staff:view
POST	/api/users/staff	staff:manage
GET	/api/users/guests	guests:view
PATCH	/api/users/:id	staff:manage or guests:manage